Meeting Notes & Transcription Platform

(Fireflies.ai Clone) — SDE Fullstack Assignment

Description

Build a functional clone of the Fireflies.ai meeting-assistant web application that replicates its design, user experience, and core post-meeting workflows.

The platform should allow users to browse a library of meetings, view interactive transcripts with speaker labels and timestamps, read AI-generated summaries and action items, search across transcripts, and experience the clean, productivity-focused interface of the original Fireflies app.

Your implementation should visually and functionally feel like a modern Fireflies workspace. The focus is on recreating the Fireflies experience and core meeting/transcript workflows rather than performing real speech-to-text — transcription and AI summaries can be mocked, seeded, or generated from uploaded transcript files.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)

Note: real audio transcription is out of scope. You may seed transcript data, accept an uploaded transcript file (e.g. .txt/.vtt/.json), or optionally call an LLM to generate summaries from existing transcript text.

Core Features (Must Have)

1. Meetings Library / Dashboard

Recreate the Fireflies meetings home view.

- List of past meetings with title, date, duration, and participants
- Search and filter meetings (by title, date, or participant)
- Sort by recency
- Navbar with profile/settings placeholders

2. Meeting / Transcript Detail View

Implement the core meeting page.

- Interactive transcript with speaker labels and timestamps
- A media player area with a seek bar (audio/video can be a placeholder or a sample file)

- Clicking a transcript line seeks the player to that timestamp (and vice versa)
- Search within the transcript with highlighted matches

3. AI Summary & Notes

- AI-generated meeting summary section
- Action items / tasks extracted from the meeting
- Key topics / outline / chapters
- Summaries can be seeded, mocked, or LLM-generated from transcript text

4. Meeting Management (CRUD)

Implement CRUD for meetings and their contents.

- Create a meeting (by uploading or pasting a transcript, or via a form)
- Edit meeting metadata (title, participants)
- Delete a meeting
- Add / edit / complete action items
- All meetings, transcripts, summaries, and action items must persist

5. Fireflies Experience

The application should closely resemble the Fireflies experience, including:

- Navigation and layout (library + detail view)
- Transcript and summary panels
- Forms, modals, search, and filters
- Notifications / toasts
- Settings placeholders

The goal is to make the application feel like Fireflies rather than a generic notes app.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Real-time bot that joins live calls
- Actual speech-to-text transcription
- Integrations (Zoom, Google Meet, calendar, CRM)
- Team / sharing & collaboration
- Real user authentication (assume a default logged-in user)

Bonus (Optional)

- Comments / highlights / soundbites on transcript segments
- Export transcript or summary (PDF / Markdown / TXT)
- Global search across all meetings
- Tags / topics and filtering by them
- LLM-powered “ask a question about this meeting” chat
- Dark mode

Important Notes

- UI Design: your application should totally resemble Fireflies's design. Study Fireflies's UI carefully before starting.
- Sample Data: seed your database. Seed several meetings with full transcripts, summaries, and action items so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including the interactive transcript and summary views
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships
Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components
Code Understanding	Ability to explain your code during evaluation

Timeline

Estimated effort: approximately 24 hours of work.

Submission Deadline: as communicated alongside this assignment.

Good luck!